

Report Builder

Self-Service Reporting — Proof of Concept

Demo scope — not the production build

D55 Consulting
August 2026

BrytEnergy

What the POC shows the client

- **Builder canvas** - drag tables from a palette, connect joins visually
- **Column picker** - choose exactly what each table contributes
- **AI assistant** - refine the report in plain language; the canvas updates live
- **Preview** - an instant sample of the output
- **Run & download** - a real CSV result to finish on

The demo proves the experience - the assistant is the star

What the POC deliberately defers

- The whole **governance & security spine** - per-customer isolation, output verifier, injection defence, query bounds
- Production infrastructure - Step Functions, DynamoDB single-table, versioned S3, IAM + Lake Formation, JWT auth
- Multi-tenant identity + admin override - the POC runs one demo user against one configured scope
- Run cancellation, deep history, presigned downloads, conversation persistence

All of this lands in the full build on green-light - not a hardened POC

POC Estimate Summary

PHASE	REQUIRED (DAYS)	TOTAL (DAYS)
Phase 1: Foundations	0.8	0.8
Phase 2: Query generation + validation	0.8	0.8
Phase 3: Backend services	1.2	1.2
Phase 4: Run + preview + download	0	0
Phase 5: Frontend	3.5	3.5
Phase 6: Demo readiness	0.5	0.5
TOTAL	6.8	6.8

Phase 1: Foundations

~0.8 days

- › Lightweight project scaffold (api/ + web/) - not the strict production repo layout
- › Shared **Report_Design** domain types - same shapes as the full spec, so they carry forward
- › Bring in the Join_Manifest + pick the minimal demo table set; define the single demo scope + fixed LIMIT

The kept pieces seed the production build later

Phase 2: Query generation + validation

~0.8 days

- › **validateDesign** - allow-listed tables/columns + manifest joins only (a correctness check, not a security gate)
- › **Query_Generator** - design → Athena SQL, scoped to the demo scope, fixed LIMIT, bound parameters
- › Report_Design serialise/deserialise round-trip

No independent verifier - that is a full-build concern

Phase 3: Backend services

~1.2 days

- › Catalog service - static / cached allow-list + Join_Manifest (no fail-closed governance)
- › Reports CRUD against a simple store (one table or local JSON)
- › **Assistant** - Bedrock Converse tool-use loop + Report_Design mutation tools
- › Optional dry-run EXPLAIN polish (not a gate)

The assistant is the centrepiece - prioritised over everything

Phase 4: Run + preview + download

~0 days

- › Run handler: generate → Athena → CSV, polled to completion (no Step Functions)
- › CSV download for a completed run
- › Synchronous bounded preview (no run queued)

Queued → Running → Complete / Failed - no Cancelled state

Phase 5: Frontend (Angular Portal feature)

~3.5 days

- › Flow-canvas library spike (ngx-xyflow vs f-flow)
- › Client Report_Design + pure graph mapping (canvas ↔ assistant in sync)
- › The demo screens: My Reports, Builder, Column picker, Assistant drawer, Run, Preview, Save

Phase 6: Demo readiness

~0.5 days

- › Seed a couple of pre-built reports + demo data
- › Script the 'ask the assistant' moment so it lands reliably
- › End-to-end run-through - smooth and repeatable in front of the client

The rehearsal that makes the demo land

Next Steps

~6.8 developer days for the POC

- › Build the POC demo from Phase 1 - prioritise the assistant, the star of the demo
- › Run against the dev twin (or a small fixture) as a single demo user / demo scope
- › Showcase to the client; gather feedback on the experience
- › On green-light, build the full production spec (with the security spine + infrastructure) - **not** a hardened POC

The full feature - requirements, design, 38-task plan, and its own estimate - lives alongside this POC in the report-builder spec.